GUIA DO PROFESSOR

LEDs: do primeiro pisca-pisca ao semáforo controlado pelo Wi-Fi

Plano de 4 aulas, fundamentação, gabarito dos desafios, solução de problemas e avaliação

---

Placas: Arduino Uno → ESP32 • **Foco:** lógica de programação com feedback físico imediato

Projeto final: semáforo + LED RGB controlados por página web local (ESP32 em modo Access Point)

Público: Ensino Fundamental II / Médio / Técnico • **Carga horária:** 4 aulas de 100 min

Espaço Cria — 2026

Sumário

1. Visão geral
2. Materiais
3. Preparação
4. Plano das aulas
5. Fundamentação técnica para o professor
6. Erros comuns e como resolver
7. Gabarito e comentários dos desafios
8. Avaliação
9. Diferenciação
10. Segurança
11. Referências

Anexo A — Todos os esquemas de montagem

Anexo B — Sketch final de referência

1. Visão geral

Esta sequência usa o LED — o atuador mais simples e mais "visível" que existe — como pretexto para ensinar **lógica de programação**: sequência, repetição, condição, funções, variáveis de estado, eventos e tempo. A montagem elétrica é deliberadamente mínima e quase não muda de uma atividade para outra; **toda a dificuldade está no raciocínio**. A progressão culmina na migração para o ESP32 e no controle por uma página web, o que introduz de forma concreta redes, HTTP e a separação entre interface e controle.

Objetivos de aprendizagem

- Explicar o papel do resistor em série e a polaridade do LED; calcular a corrente.
- Escrever programas com `setup`/`loop`, `pinMode`, `digitalWrite`, `digitalRead`, `delay`, `if`, `for` e funções.
- Distinguir **estado** de **evento** (borda) e implementar debounce.
- Modelar um problema como **máquina de estados** e implementá-la com e sem `delay()` (`millis()`).
- Explicar PWM e usar `analogWrite`/LEDC para misturar cores; usar `analogRead` + `map`.
- Migrar um projeto entre plataformas identificando o que muda.
- Descrever o fluxo cliente-servidor (navegador → HTTP → ESP32 → LED) e modificar uma página web simples.
- Depurar sistematicamente com o Monitor Serial.

Competências relacionadas (BNCC)

- **Pensamento computacional** (EF/EM): decomposição, algoritmos, abstração (funções), reconhecimento de padrões (tabela de estados).
- **Cultura digital**: entender o que há por trás de um app que controla um aparelho.
- **Matemática**: proporcionalidade (`map`), porcentagem (PWM), resto da divisão (ciclos), tempo.
- **Ciências**: circuito elétrico, corrente, resistência, cor como mistura de luz.

Pré-requisitos

Nenhum. A Aula 1 começa do zero (o que é um microcontrolador). Alunos que já piscaram um LED avançam direto para os desafios.

2. Materiais

Por grupo (2–3 alunos)

Item	Qtd	Custo aprox. (R$)	Observação
Arduino Uno + cabo USB	1	40–90	Aulas 1–3 (ou usar o ESP32 desde o início — ver §5)
ESP32 Dev Module (30 pinos) + cabo micro-USB de dados	1	35–60	Aula 4
Protoboard 830 pontos	1	15–25	a de 400 pontos serve para as aulas 1–3

Item	Qtd	Custo aprox. (R$)	Observação
LEDs 5 mm: 2 vermelhos, 1 amarelo, 1 verde	4	0,30 cada	+ alguns reservas (invertem e queimam)
LED RGB 5 mm cátodo comum	1	1–2	verificar polaridade do lote
Resistores 220 Ω	8	0,10 cada	330 Ω também serve
Push-button 12 mm	2	0,50	
Potenciômetro 10 kΩ	1	2–4	
Jumpers macho-macho	20	10 (kit)	

Para a turma

- Multímetro (1 a cada 2 grupos): testar LED (função diodo) e continuidade.
- Celulares dos alunos ou 1 notebook por grupo com Wi-Fi para a Aula 4.
- Projetor: as figuras deste guia (`aula-leds/esquemas/saida/*.png`) em alta resolução para projetar.
- Caixa de LEDs/resistores reserva e alguns Unos extras — um pino queimado no meio da aula não pode parar o grupo.

3. Preparação

1. Arduino IDE 2 com o core **esp32 by Espressif** (`https://espressif.github.io/arduino-esp32/package_esp32_index.json`) instalado nos PCs. Nenhuma biblioteca extra é necessária (WiFi e WebServer vêm com o core).
2. Drivers USB-serial **CP210x** e **CH340** instalados (a maioria dos ESP32 e clones de Uno usa um deles).
3. Copie `aula-leds/codigo/` para as máquinas. Todos os sketches foram **compilados e verificados** (Uno: 01, 01b, 02, 03, 03b, 04, 04b; ESP32: 05, 06).
4. Teste o sketch 06 em um ESP32 antes da Aula 4: conecte o celular à rede `Semaforo-ESP32` e abra `http://192.168.4.1`. Deixe esse ESP32 pronto como demonstração de abertura do curso.
5. Separe os LEDs RGB e confirme se são de cátodo comum (comum no GND) — o código tem a chave `ANODO_COMUM`.
6. Os esquemas de montagem foram feitos no **Fritzing** e estão em `aula-leds/esquemas/saida/*.fzz` (editáveis) e `*.png`. Regenerar: `python esquemas.py`.
7. Linha de comando (opcional): `arduino-cli compile --fqbn esp32:esp32:esp32 06_esp32_web` / `arduino-cli upload -p COM3 ...`.

4. Plano das aulas

Cada aula tem 100 min. A coluna **Evidência** indica o que observar para avaliação formativa. Em todas as aulas: comece com uma **pergunta-problema**, deixe os alunos errarem no código antes de dar a resposta, e termine com um *ticket de saída*.

Aula 1 — Ligar um LED e fazê-lo obedecer

Tempo	Atividade	Professor	Alunos	Evidência
10'	Abertura	Demonstra o projeto final (celular controlando semáforo). “Como o celular fala com um LED?” Anota hipóteses	Levantam hipóteses	Hipóteses no quadro
15'	O que é Arduino/ESP32	Mostra as duas placas, pinos, `setup`/`loop`. Apresenta o LED e o resistor (cálculo da corrente no quadro)	Anotam; identificam pernas do LED	—
15'	Montagem 1 + sketch 01	Circula conferindo polaridade e resistor. Primeiro upload: conferir placa/porta	Montam, gravam, veem piscar	LED piscando
15'	Experimente 1–3	Provoca com a pergunta da fusão de imagens (delay 10 ms)	Alteram delays, discutem	Respostas orais
15'	Desafios 1.1–1.4	Não dá resposta; pede para **explicar em português** antes de codificar	Programam SOS/batimento	Sketch funcionando
10'	Montagem 2 + sketch 01b	Explica INPUT_PULLUP (apertado = 0) e mostra a Versão A	Montam o botão	LED segue o botão
15'	Borda e toggle	Pergunta “por que a versão B precisa lembrar o estado anterior?” — demonstra sem o `if` de borda	Testam a Versão B	Explicam borda vs. estado
5'	Fechamento	Ticket: “o que acontece se tirar o `delay(50)`?”	Respondem	Ticket de saída

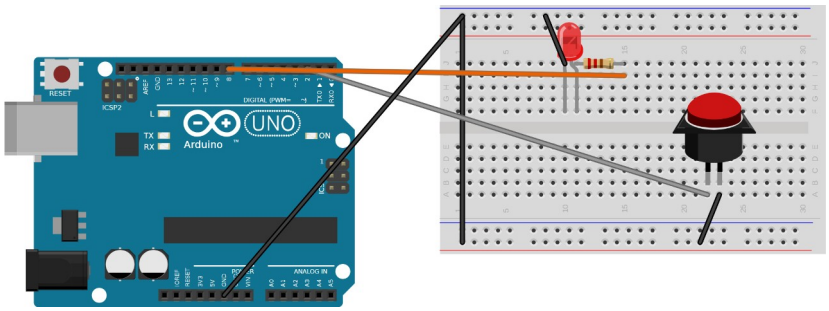


Figura 1 — Aula 1 — montagem final (LED no pino 8, botão no pino 2). Projetar durante a montagem.

Aula 2 — Semáforo

Tempo	Atividade	Professor	Alunos	Evidência
10'	Problema	"Escrevam no papel o ciclo de um semáforo com tempos." Introduz a ideia de estado	Escrevem a sequência	Tabela de estados no papel
15'	Montagem 3 LEDs	Confere 3 resistores e cátodos no GND	Montam	Checklist
15'	Sketch 02 + funções	Mostra `mostrar(cor)` e a comparação que vira HIGH/LOW. Pede para reescrever sem a função: 9 linhas × 3	Gravam; comparam	Explicam a função
10'	Experimente 1–3	Semáforo de madrugada; "por que `mostrar` não acende dois?"	Alteram	Respostas
20'	O problema do delay	**Demonstração-chave**: sketch 02 + botão: aperta e nada acontece. Explica `millis()` com um relógio de parede	Testam o sketch 03	Percebem o loop livre
20'	Botão de pedestre (03b)	Enfatiza "anotar o pedido" vs. "reagir ao pedido"	Montam o botão; testam	Pedido atendido corretamente
5'	Desafios 2.x	Distribui por nível: 2.1 (todos), 2.3 (avançados). Pede a **tabela de estados antes do código**	Escolhem	—
5'	Fechamento	Ticket: "dê um exemplo de máquina de estados fora da eletrônica"	Respondem	Ticket

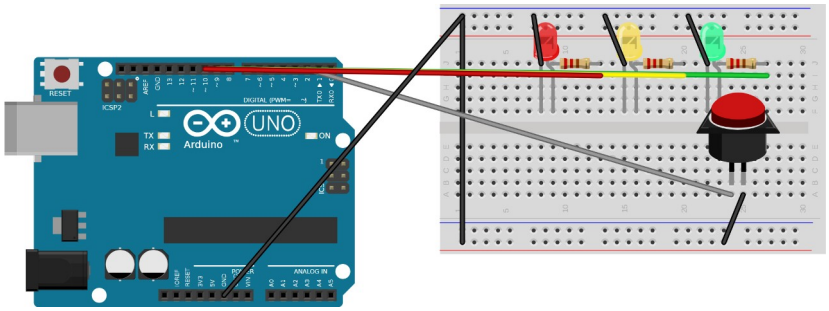


Figura 2 — Aula 2 — semáforo com botão de pedestre.

Aula 3 — LED RGB

Tempo	Atividade	Professor	Alunos	Evidência
10'	Cor é luz	Lanterna com celofane R, G, B sobrepostos (ou vídeo). "O amarelo do LED é o mesmo do lápis?"	Observam	—
15'	PWM	Sketch 01 com delay(1)/delay(3): brilho médio! Depois	Testam brilhos	Explicam PWM

Tempo	Atividade	Professor	Alunos	Evidência
		`analogWrite`. Osciloscópio ou plotter serial se houver		
15'	Montagem RGB	Identifica o comum (perna longa) e a ordem R-G-B do lote; teste rápido com 5 V + resistor	Montam	3 cores puras acendem
10'	Sketch 04	Discute mistura aditiva; pede uma cor nova por grupo	Gravam; criam cores	Tabela de cores preenchida
15'	Pot + botão (04b)	Explica `map` e a variável `modo`; desenha as 3 curvas de `corPorMatiz`	Montam pot e botão	Pot varre o arco-íris
30'	Desafios 3.x	3.2 é obrigatório (reusa a máquina de estados); 3.4 prepara a ideia de gravar/reproduzir	Programam	Sketch + explicação
5'	Fechamento	Ticket: “por que (255,255,0) parece esverdeado e (255,60,0) é amarelo?”	Respondem	Ticket

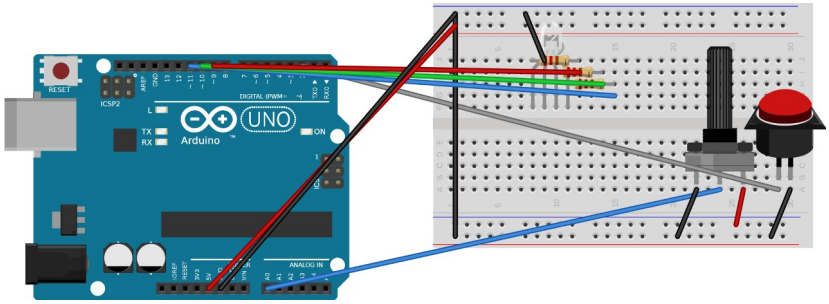


Figura 3 — Aula 3 — LED RGB, potenciômetro e botão.

Aula 4 — ESP32 e controle pela web

Tempo	Atividade	Professor	Alunos	Evidência
10'	Migração	Tabela Uno × ESP32; “o que precisa mudar no sketch 04b?” — alunos listam antes de ver o 05	Listam mudanças	Lista
20'	Montagem ESP32 + sketch 05	Confere pot no 3V3; auxilia com drivers/BOOT	Montam; gravam	Semáforo + RGB funcionando
15'	Rede e HTTP	Explica AP, IP, rota, pedido/resposta com uma dramatização (aluno-navegador pede ao aluno-ESP32)	Participam	—
15'	Sketch 06	Conecta o celular; mostra o Serial imprimindo cada pedido; abre `/semaforo?cor=verde` direto na barra do navegador	Conectam seus celulares	Página controlando LEDs

Tempo	Atividade	Professor	Alunos	Evidência
10'	Anatomia da página	HTML (o que aparece) / JS `fetch` (o que faz) / rotas no ESP32 (quem responde). Muda o título ao vivo	Alteram texto/cores da página	Página personalizada
20'	Desafios 4.x	4.1 e 4.2 para todos; 4.4 para discutir conflito local × remoto	Programam	Sketch
10'	Apresentação e fechamento	Cada grupo demonstra; rubrica (§8)	Apresentam	Rubrica

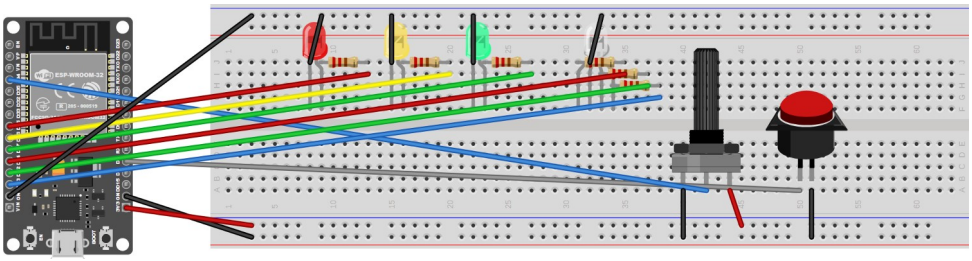


Figura 4 — Aula 4 — montagem completa no ESP32.

5. Fundamentação técnica para o professor

LED e resistor

O LED conduz a partir de ~1,8 V (vermelho) a ~3,2 V (azul/branco) e deve ficar abaixo de 20 mA. Com 220 Ω em 5 V: (5 − 2)/220 ≈ 14 mA. No ESP32 (3,3 V): (3,3 − 2)/220 ≈ 6 mA — acende mais fraco; 100 Ω daria ~13 mA, mas 220 Ω é mais seguro para quem inverte ligações. O pino do Uno aguenta 40 mA (ideal < 20 mA); o do ESP32, ~12 mA — **nunca** LED sem resistor. LED invertido não queima, só não acende.

INPUT_PULLUP, borda e debounce

O resistor interno (~20–50 kΩ) puxa a entrada a VCC; o botão a leva a GND. Sem pull-up a entrada fica **flutuando** e lê valores aleatórios — causa clássica de "LED que liga sozinho". Um botão mecânico oscila por 1–10 ms ao fechar (bounce); sem debounce um toque vira 3–10 eventos. O `delay(50)` é a solução didática; a robusta usa `millis()` (o sketch 06 do ESP32 mantém o `delay(50)` porque 50 ms não atrapalham o servidor).

delay() × millis()

`delay` bloqueia: o processador conta ciclos e ignora tudo. `millis()` retorna um contador de 32 bits (estoura a cada 49,7 dias; a subtração `millis() - inicio` continua correta mesmo no estouro, por aritmética não sinalizada). O padrão "se `millis() - inicio >= intervalo`, faça e atualize `inicio`" é a base de todo programa que faz várias coisas ao mesmo tempo sem sistema operacional. Insista nele: é o conceito mais transferível do curso.

PWM

No Uno, `analogWrite` usa os timers em ~490 Hz (980 Hz nos pinos 5 e 6), 8 bits. No ESP32, o periférico LEDC tem 16 canais; no core 3.x `ledcAttach(pino, freq, bits)` aloca um canal automaticamente. Frequência de 5 kHz evita cintilação visível em câmera. Percepção de brilho é logarítmica: 128 parece bem mais que "metade" — bom gancho para discutir escalas.

Wi-Fi, AP e HTTP

Em modo **Access Point** o ESP32 é o roteador (IP fixo 192.168.4.1, até 4 clientes) — evita depender da rede da escola e de senhas. Em modo estação (`WiFi.begin`) ele entra na rede existente e o IP aparece no Serial. O `WebServer` responde a `GET`: rota + parâmetros na URL. A página usa `fetch()` (AJAX) para não recarregar. Não há segurança alguma — qualquer um na rede controla o semáforo; é um ótimo ponto de discussão.

Se a escola só tiver ESP32

Toda a sequência funciona no ESP32 desde a Aula 1: use os pinos 25/26/27, 14/12/13, 4 e 34, `ledcAttach` em vez de `analogWrite` e `0..4095` no `map`. O sketch 05 já é o 03b + 04b migrados; basta apresentá-lo por partes.

6. Erros comuns e como resolver

Sintoma	Causa provável	Solução
LED não acende	Invertido; resistor/fio na linha errada da protoboard; pino errado no código	Inverter o LED; conferir se resistor e LED estão na mesma coluna (linhas A–E são um grupo, F–J outro)
LED acende fraco/esquenta a placa	Sem resistor ou resistor de valor errado (22 Ω)	Ler as faixas: vermelho-vermelho-marrom
Botão "aperta sozinho" / LED aleatório	Pino flutuando: sem INPUT_PULLUP ou botão fora do GND	`pinMode(pino, INPUT_PULLUP)`; outro lado no GND
Um toque vira vários	Sem debounce ou sem detecção de borda	`delay(50)` após a borda; guardar `botaoAntes`
Botão de pedestre não responde	Sketch com `delay()`	Usar 03b (millis)
Cor do RGB invertida (255 apaga)	Ânodo comum	`ANODO_COMUM = true` e comum no 5 V/3V3
RGB: "amarelo" esverdeado	Verde é mais eficiente que vermelho	Reduzir G (255, 60–150, 0)
`analogWrite` não regula brilho	Pino sem PWM (Uno: só 3, 5, 6, 9, 10, 11)	Trocar de pino
Porta COM não aparece	Cabo só de carga; driver ausente	Cabo de dados; instalar CP210x/CH340
"Failed to connect to ESP32"	Auto-reset falhou	Segurar BOOT ao iniciar o upload
Upload OK mas nada acontece no ESP32	Monitor Serial na velocidade errada / placa errada	115200; ESP32 Dev Module
Pot lê sempre 4095 no ESP32	Pot no 5 V (saturado) ou GPIO de ADC2 com Wi-Fi	Pot no 3V3; usar GPIO 32–39
Rede `Semaforo-ESP32` não aparece	Sketch 05 (sem Wi-Fi) gravado; ESP32 sem energia suficiente	Gravar 06; usar porta USB boa / cabo curto
Página abre mas botões não fazem nada	Celular saiu da rede do ESP32 (voltou para dados móveis)	Desligar dados móveis; reconectar
Página demora ou "trava"	`delay()` no loop	Nenhum `delay` longo com `handleClient()`
Compila no Uno, não no ESP32 (`analogWrite`/`ledcSetup`)	API do core 3.x	Usar `ledcAttach`/`ledcWrite` (core ≥ 3.0)

7. Gabarito e comentários dos desafios

Aula 1

- **1.1** `liga; delay(100); desliga; delay(100); liga; delay(100); desliga; delay(700);` — bom momento para introduzir função `pisca(ms)`.
- **1.2** `void curto(){ on; delay(200); off; delay(200);}` `void longo(){ on; delay(600); off; delay(200);}` e `for (i<3) curto(); for (i<3) longo(); for (i<3) curto(); delay(1400);`. Avaliar quem percebe a repetição e usa `for`.
- **1.3** `int t = 500; loop: pisca(t); t -= 20; if (t < 10) t = 500;` — variável que muda a cada volta = **estado**.
- **1.4** Solução A: `for` no `setup()` (5 piscadas) e `loop()` vazio. Solução B: `static int n = 0; if (n < 5) { pisca(); n++; }`. Discutir as duas: a B permite "rearmar" com um botão.
- **1.5** `contador = contador % 5 + 1;` na borda; depois `for (i < contador) pisca(150);`.
- **1.6** `if (digitalRead(BOTAO) == LOW) pisca(50); else digitalWrite(LED, HIGH);` — aqui **estado**, não borda.
- **1.7** Guardar `ultimoToque = millis()` na borda; se `millis() - ultimoToque < 500` na borda seguinte → liga.
- **1.8** `delay(random(1000, 4000)); liga; t0 = millis(); while (digitalRead(BOTAO) == HIGH); Serial.println(millis() - t0);` — o `while` bloqueante é aceitável aqui e rende discussão.

Aula 2

- **2.1** Novo pino no vetor ou `digitalWrite(PEDESTRE, faseAtual == 2)` em `mostrar`; piscar: `if (faseAtual == 2 && restante < 2000) digitalWrite(PEDESTRE, (millis() / 250) % 2)`.
- **2.2** Medir tempo de botão pressionado com `millis()`; variável `modoNoturno`; no `loop`, se noturno: amarelo = `(millis()/500) % 2`, demais apagados.
- **2.3** Tabela: (V1,R2) → (A1,R2) → (R1,R2) → (R1,V2) → (R1,A2) → (R1,R2) → … 6 estados; vetor de pares. O "vermelho duplo" de segurança entre as trocas é o ponto de avaliação.
- **2.4** `int restante = (FASE_MS[2] - (millis() - inicioFase)) / 1000;` imprimir só quando mudar (guardar o último impresso) — evita spam no Serial.
- **2.5** `if (Serial.available()) { char c = Serial.read(); switch (c) { case 'V': ... } }` — introduz `switch` e parsing simples.

Aula 3

- **3.1** `if (pot < 512) cor(map(pot,0,511,0,255), map(pot,0,511,0,255), 255); else cor(255, map(pot,512,1023,255,0), map(pot,512,1023,255,0));`.
- **3.2** Reusar `FASE_*` com um vetor de cores `{ {0,255,0}, {255,60,0}, {255,0,0} }`. Amarelo (255,255,0) parece verde porque o LED verde é 2–3× mais eficiente que o vermelho.
- **3.3** Vetor de 6 cores; na borda: `for (i<10) { corIndex(random(6)); delay(60); }` depois fixa uma.
- **3.4** Vetor `int cores[3][3]`, índice de gravação na borda; modo de reprodução quando `n == 3`. Mesma estrutura do "teach & repeat" de robótica.

- **3.5** Alvo aleatório a cada 3 s; a cada 10 ms `atual += (alvo - atual) * 0.05` (ou passo de 1 por componente) — interpolação linear.

Aula 4

- **4.1** Rota `/estado` → `servidor.send(200, "text/plain", corSemaforo);` na página `setInterval(() => fetch('/estado').then(r => r.text()).then(t => estado.textContent = t), 1000)`.
- **4.2** Botões com `onclick="rgbfix(255,0,0)"` chamando `fetch('/rgb?r=...')`.
- **4.3** `FASE_MS` deixa de ser `const`; rota `/tempo` lê `servidor.arg("verde").toInt()` com validação (500–20000 ms).
- **4.4** Guardar `ultimoPot = millis()` quando a leitura muda mais que um limiar; na rota `/rgb`, se `millis() - ultimoPot < 5000`, responder "pot no controle" sem aplicar. Discutir quem deve ter prioridade e por quê.
- **4.5** No segundo ESP32: `WiFi.begin("Semaforo-ESP32", "12345678")` e `HTTPClient http; http.begin("http://192.168.4.1/semaforo?cor=verde"); http.GET();`.

8. Avaliação

Rubrica do projeto final

Critério	Iniciante (1)	Em desenvolvimento (2)	Proficiente (3)
Funcionamento	Semáforo ou RGB funcionam só parcialmente	Tudo funciona pela página, com ajuda	Tudo funciona pela página **e** pelos controles físicos; sem travar
Lógica e código	Reproduz o sketch	Explica cada bloco; resolveu 1 desafio por aula	Explica **por que** (borda, millis, PWM, rotas); resolveu desafios avançados e modificou com propósito
Máquina de estados	Não relaciona código com estados	Desenha a tabela de estados do semáforo	Cria uma nova máquina (cruzamento, noturno) a partir da tabela
Depuração	Depende do professor	Usa o Serial para localizar o problema	Diagnostica pela tabela sintoma→causa e resolve sozinho
Comunicação	Um aluno faz tudo	Papéis definidos	Todos explicam uma parte; demonstração clara para a turma

Formativa

- Tickets de saída de cada aula (listados nos planos).
- "Explique em português antes de programar": pedir a sequência de passos escrita para cada desafio.
- Sketches dos desafios com comentário no topo explicando a ideia.

- Autoavaliação: sei fazer sozinho / com ajuda / ainda não sei, para os 8 objetivos do §1.

9. Diferenciação

- **Ritmos diferentes**: cada aula tem desafios em escada (1.1 → 1.8). Nenhum grupo fica sem tarefa; nenhum precisa fazer todos.
- **Avançados**: Aula 2 → cruzamento (2.3); Aula 3 → lâmpada de humor (3.5); Aula 4 → dois ESP32 (4.5) ou mudar a página com CSS.
- **Dificuldade**: fornecer os sketches com `// <<< MUDE AQUI` nos pontos de alteração; par com um colega; começar pelos "Experimente".
- **Sem Uno**: tudo no ESP32 (§5). **Sem ESP32**: Aulas 1–3 são completas em si; a Aula 4 pode virar "semáforo controlado pelo Serial" (desafio 2.5).

10. Segurança

- Placas alimentadas só por USB nas aulas 1–4 (não há fonte externa).
- LEDs sempre com resistor; nunca ligar pinos entre si ou ao 5 V.
- Desligar o USB para mexer na montagem.
- Rede `Semaforo-ESP32` é aberta ao alcance da sala — combinar regras de uso (um grupo por vez controla o seu).

11. Referências

- Arduino — referência da linguagem: https://www.arduino.cc/reference/pt/
- Arduino — Blink, Button, Debounce, BlinkWithoutDelay, Fade (Arquivo → Exemplos → Basics/Digital).
- ESP32 Arduino Core — LEDC e WebServer: https://docs.espressif.com/projects/arduino-esp32/
- Fritzing — esquemas em `aula-leds/esquemas/`.
- Sketches deste curso: `aula-leds/codigo/` (01 a 06).

Anexo A — Todos os esquemas de montagem

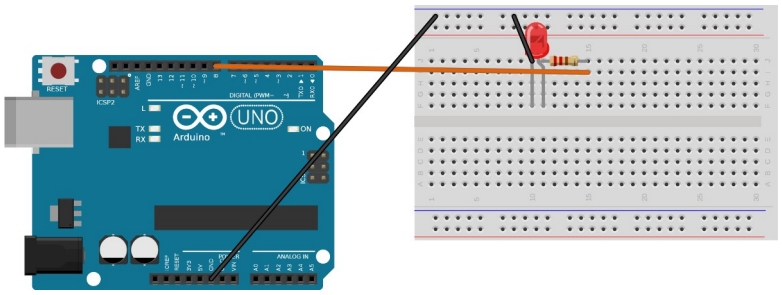


Figura 5 — Aula 1 — LED (sketch 01).

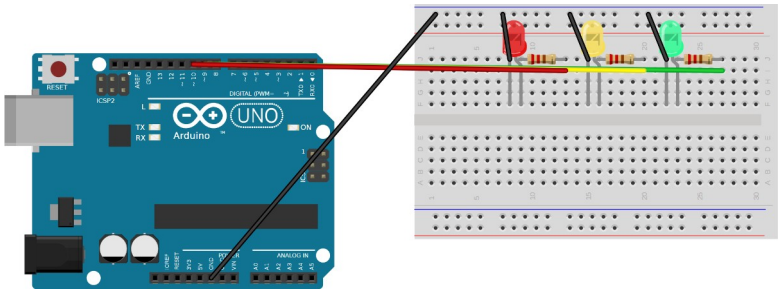


Figura 6 — Aula 2 — semáforo (sketches 02 e 03).

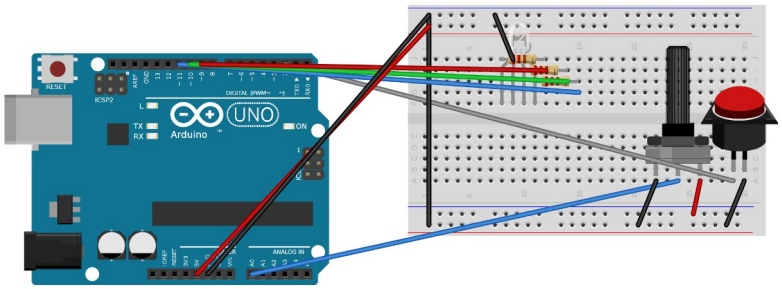


Figura 7 — Aula 3 — RGB + pot + botão (sketches 04 e 04b).

Anexo B — Sketch final de referência

06_esp32_web.ino

```
/*
  06 - Controle pelo Wi-Fi: pagina web local para o semaforo e o LED RGB
  Placa: ESP32 Dev Module (core esp32 3.x)   Ligacoes: iguais ao sketch 05

  Como funciona:
    1. O ESP32 entra na rede Wi-Fi (ou cria a sua propria, se MODO_AP = true).
    2. Ele roda um SERVIDOR WEB: quem acessar http://<ip>/ recebe a pagina HTML abaixo.
    3. Os botoes da pagina chamam enderecos como /semaforo?cor=verde  e  /rgb?r=255&g=0&b=0
    4. O ESP32 le os parametros e acende os LEDs. Tudo sem delay(), para nunca travar o servidor.

  Abra o Monitor Serial (115200) para ver o endereco IP. Celular e ESP32 na MESMA rede.
*/
#include <WiFi.h>
#include <WebServer.h>

// ---------- rede ----------
const bool MODO_AP = true;                 // true: o ESP32 cria a rede "Semaforo-ESP32" (senha abaixo)
const char* SSID_AP = "Semaforo-ESP32";    // false: entra na rede da escola (preencha abaixo)
const char* SENHA_AP = "12345678";
const char* SSID_ESCOLA = "NOME-DA-REDE";
const char* SENHA_ESCOLA = "SENHA-DA-REDE";

// ---------- pinos ----------
```

```
const int VERMELHO = 25, AMARELO = 26, VERDE = 27;
const int PINO_R = 14, PINO_G = 12, PINO_B = 13;
const int BOTAO = 4, PINO_POT = 34;        // botao fisico: avanca o semaforo; pot: brilho geral do RGB
bool botaoAntes = HIGH;

WebServer servidor(80);

// estado atual (o que a pagina mostra e o que os LEDs fazem)
String corSemaforo = "auto";               // "auto", "verde", "amarelo", "vermelho", "off"
int corR = 0, corG = 0, corB = 128;
const int FASE_PINO[] = { VERDE, AMARELO, VERMELHO };
const unsigned long FASE_MS[] = { 4000, 1500, 4000 };
int faseAtual = 0;
unsigned long inicioFase = 0;

// ---------- pagina HTML (fica na memoria flash) ----------
const char PAGINA[] PROGMEM = R"HTML(
<!DOCTYPE html><html lang="pt-br"><head><meta charset="utf-8"><meta name="viewport" content="width=device-
width,initial-scale=1">
<title>Semaforo ESP32</title>
<style>
 body{font-family:sans-serif;background:#111;color:#eee;text-align:center;margin:0;padding:16px}
 h1{font-size:1.4em} .card{background:#222;border-radius:12px;padding:16px;margin:12px auto;max-
width:360px}
 button{font-size:1.1em;padding:12px 18px;margin:6px;border:0;border-radius:8px;color:#fff;cursor:pointer}
 .vd{background:#2a2} .am{background:#ca0} .vm{background:#c22} .au{background:#448} .off{background:#555}
 input[type=color]{width:120px;height:60px;border:0;background:none}
 #estado{opacity:.8}
</style></head><body>
<h1>Semaforo + RGB pelo Wi-Fi</h1>
<div class="card"><h2>Semaforo</h2>
 <button class="vd" onclick="sem('verde')">Verde</button>
 <button class="am" onclick="sem('amarelo')">Amarelo</button>
 <button class="vm" onclick="sem('vermelho')">Vermelho</button><br>
 <button class="au" onclick="sem('auto')">Automatico</button>
 <button class="off" onclick="sem('off')">Apagar</button>
 <p id="estado">estado: %SEMAFORO%</p></div>
<div class="card"><h2>LED RGB</h2>
 <input type="color" id="cor" value="%COR%" oninput="rgb(this.value)">
 <p>toque na cor para escolher</p></div>
<script>
 function sem(c){fetch('/semaforo?
cor='+c).then(r=>r.text()).then(t=>document.getElementById('estado').textContent='estado: '+t)}
 function rgb(hex){const
r=parseInt(hex.substr(1,2),16),g=parseInt(hex.substr(3,2),16),b=parseInt(hex.substr(5,2),16);
   fetch('/rgb?r='+r+'&g='+g+'&b='+b)}
</script></body></html>
)HTML";

// ---------- LEDs ----------
void mostrar(int pino) {
  digitalWrite(VERMELHO, pino == VERMELHO);
  digitalWrite(AMARELO,  pino == AMARELO);
  digitalWrite(VERDE,    pino == VERDE);
}
void aplicarSemaforo() {
  if (corSemaforo == "verde")         mostrar(VERDE);
  else if (corSemaforo == "amarelo")  mostrar(AMARELO);
  else if (corSemaforo == "vermelho") mostrar(VERMELHO);
  else if (corSemaforo == "off")      mostrar(-1);
  // "auto": quem cuida e o loop() com millis()
}
int brilho = 255;                                  // 0..255, vem do potenciometro
void cor(int r, int g, int b) {                    // aplica o brilho do pot a cor escolhida na pagina
  ledcWrite(PINO_R, r * brilho / 255);
  ledcWrite(PINO_G, g * brilho / 255);
  ledcWrite(PINO_B, b * brilho / 255);
}

// ---------- rotas do servidor ----------
void paginaInicial() {
  String html = FPSTR(PAGINA);
  html.replace("%SEMAFORO%", corSemaforo);
  char hex[8]; snprintf(hex, sizeof(hex), "#%02x%02x%02x", corR, corG, corB);
```

```
  html.replace("%COR%", hex);
  servidor.send(200, "text/html; charset=utf-8", html);
}
void rotaSemaforo() {
  if (servidor.hasArg("cor")) corSemaforo = servidor.arg("cor");
  aplicarSemaforo();
  Serial.println("semaforo -> " + corSemaforo);
  servidor.send(200, "text/plain", corSemaforo);
}
void rotaRgb() {
  corR = servidor.arg("r").toInt();
  corG = servidor.arg("g").toInt();
  corB = servidor.arg("b").toInt();
  cor(corR, corG, corB);
  Serial.printf("rgb -> %d %d %d\n", corR, corG, corB);
  servidor.send(200, "text/plain", "ok");
}

void setup() {
  Serial.begin(115200);
  pinMode(VERMELHO, OUTPUT); pinMode(AMARELO, OUTPUT); pinMode(VERDE, OUTPUT);
  pinMode(BOTAO, INPUT_PULLUP);
  analogReadResolution(12);
  ledcAttach(PINO_R, 5000, 8); ledcAttach(PINO_G, 5000, 8); ledcAttach(PINO_B, 5000, 8);
  cor(corR, corG, corB);

  if (MODO_AP) {
    WiFi.softAP(SSID_AP, SENHA_AP);
    Serial.print("Rede criada: "); Serial.print(SSID_AP);
    Serial.print("  ->  acesse http://"); Serial.println(WiFi.softAPIP());
  } else {
    WiFi.begin(SSID_ESCOLA, SENHA_ESCOLA);
    Serial.print("Conectando");
    while (WiFi.status() != WL_CONNECTED) { delay(500); Serial.print("."); }
    Serial.print("\nConectado!  ->  acesse http://"); Serial.println(WiFi.localIP());
  }

  servidor.on("/", paginaInicial);
  servidor.on("/semaforo", rotaSemaforo);
  servidor.on("/rgb", rotaRgb);
  servidor.begin();
  inicioFase = millis();
}

void loop() {
  servidor.handleClient();                      // atende quem acessou a pagina
  if (corSemaforo == "auto" && millis() - inicioFase >= FASE_MS[faseAtual]) {
    faseAtual = (faseAtual + 1) % 3;
    mostrar(FASE_PINO[faseAtual]);
    inicioFase = millis();
  }
  // botao fisico: forca a proxima cor (a pagina passa a mostrar esse estado)
  bool botaoAgora = digitalRead(BOTAO);
  if (botaoAntes == HIGH && botaoAgora == LOW) {
    if (corSemaforo == "verde") corSemaforo = "amarelo";
    else if (corSemaforo == "amarelo") corSemaforo = "vermelho";
    else corSemaforo = "verde";
    aplicarSemaforo();
    delay(50);
  }
  botaoAntes = botaoAgora;
  // potenciometro: brilho geral do RGB (le a cada 100 ms para nao "tremer")
  static unsigned long ultimoPot = 0;
  if (millis() - ultimoPot > 100) {
    ultimoPot = millis();
    int novo = map(analogRead(PINO_POT), 0, 4095, 0, 255);
    if (abs(novo - brilho) > 3) { brilho = novo; cor(corR, corG, corB); }
  }
}
```